

CNN Architecture

1. CNN Architecture Overview

A CNN architecture can generally be divided into two main components:

- **Feature Extractor (Base or Body):** This part is responsible for extracting features from the input data. It is composed of Convolutional layers and Pooling layers.
- **Classifier (Head):** This part performs the actual task, such as classification or regression. It is typically composed of Fully Connected layers (also called Dense layers).
- A popular example is VGG-16, which is a CNN with 16 layers.

The three main types of layers in a CNN are:

1. Convolutional Layer
2. Pooling Layer
3. Fully Connected Layer (Dense Layer)

2. The Convolutional Layer

The convolutional layer is the most important layer where most computations occur.

2.1 The Convolution Operation

The basic computation is called **convolution**, which is essentially a Sum of Product (SOP) operation.

1. **Inputs:** A convolution operation takes an input image (or feature map) and a smaller matrix called a kernel or filter (e.g., 3x3).
2. **Sliding Window:** The kernel is slid across the entire input. The area of the input covered by the kernel at any instant is called the receptive field.
3. **Computation:** The operation involves element-wise multiplication of the kernel coefficients and the corresponding pixel values in the receptive field, followed by summing the results.
4. **Output:** The output for a single position/instance is a single scalar value (1x1).

2.2 Activation Function

- **Purpose:** The convolution operation (sum of products) is a linear operation. An activation function is necessary to introduce non-linearity to enable the model to approximate complex patterns and features (like shapes or textures).
- **Common Function:** The ReLU (Rectified Linear Unit) is a commonly used non-linear activation function in modern CNNs..
- **Activation Map:** By performing the convolution and activation for all possible positions across the input, the layer produces a 2D map of scalar values called the activation map.

2.3 Handling Multiple Channels and Filters

- **Channels:** If the input image has C channels (e.g., $C=3$ for RGB), the filter must also have C channels to match the input volume and perform the sum of product operation.
- **Multiple Filters:** A convolutional layer uses multiple filters (e.g., N_F filters). Each filter learns a specific feature, like a horizontal or vertical edge.
- **Output Depth:** The channel dimension of the output activation map is determined by the number of filters used. For N_F filters, the output is $W \times H \times N_F$.

2.4 Output Dimension and Parameters

The output dimension (N_{out}) is calculated as:

$$N_{out} = \lfloor \frac{N_{in} - F + 2P}{S} \rfloor + 1$$

- N_{in} : Input dimension (Height or Width)
- F : Filter/Kernel dimension
- P : Padding (e.g., Zero Padding)
- S : Stride length (step size)

Trainable Parameters

The total number of trainable parameters (weights and biases) in a convolutional layer is:

Total Parameters = (Number of Filters) × (Filter Dimensions × Input Channels) + (Number of Filters). *One bias term is added for each filter.*

2.5 The Pooling Layer

Pooling layers are used to downsample the feature maps.

- **Purpose:** To reduce the spatial dimensions (height and width) of the input. This helps to reduce computational complexity and the number of parameters, making the model more efficient.
- **Key Characteristic:** A pooling layer has no trainable parameters. It just performs a fixed operation.
- **Common Types:**
 - **Max Pooling:** This is the most common type. It slides a filter (e.g., 2×2) over the input and outputs only the maximum value from the receptive field.
 - **Average Pooling:** This outputs the average value from the receptive field.
- **Output Size:** The output size formula is similar to convolution's, but typically without padding ($P=0$).

2.6 The Fully Connected (Dense) Layer

This layer forms the classifier (head) of the CNN.

- **Operation:** It is a standard Multilayer Perceptron (MLP) or Artificial Neural Network (ANN). Every neuron in the layer is connected to every neuron in the next layer, which is why it's called dense.
- **Flattening:** Before feeding data into the first FC layer, the 3D output from the last pooling layer (e.g., 7x7x512) must be flattened into a 1D vector.
 - This is a reshape operation, which just rearranges the data without changing its values, as opposed to resizing, which involves interpolation.
- **Purpose:** While convolutional layers extract local and hierarchical features, the dense layers integrate this information to get a "global picture" and make the final classification decision.
- **Final Layer:** The last layer in a classifier has a number of neurons equal to the number of classes (e.g., 1000 neurons for the ImageNet dataset). It uses a Softmax activation function to convert the network's raw scores into a probability distribution.

3. Hierarchical Feature Learning

CNNs are effective because they learn features in a hierarchical manner:

- **Early Layers:** These layers capture simple patterns or generic features like edges (horizontal, vertical, diagonal lines) and contours.
- **Later Layers:** These layers take the simple features from the previous layer as input and combine them to learn increasingly complex patterns (e.g., combining edges to detect shapes like a rectangle). This process moves from learning local information (small receptive field) to gradually inferring a more global picture through layer-wise processing.

4. CNN Training

CNN training, like other neural networks, involves a forward pass and a backward pass to update parameters.

- **Forward Pass:** The input image is passed through all layers to produce the class probabilities at the output.
- **Calculate Loss:** The prediction is compared to the *actual* target (the "ground truth" label). The error is calculated using a Loss Function (e.g., Cross-Entropy Loss for classification).
- **Backward Pass (Backpropagation):** The loss is then propagated backward through the network to calculate the partial derivatives ($\frac{\partial L}{\partial W}$) and update the weights and biases using an optimizer like Stochastic Gradient Descent.
 - **Fully Connected Layers:** The update process is the same as in an ANN/MLP.
 - **Convolutional Layers:** The inverse operation of convolution, called deconvolution (or transposed convolution), is used to calculate the gradients during backpropagation.

5. Transfer Learning and Fine-Tuning

These techniques are used when the target dataset is **small** and insufficient to train a complex CNN model from scratch, which could lead to poor learning or generalization.

Transfer Learning

1. **Process:** Start with a CNN pre-trained on a massive, similar dataset (e.g., ImageNet).
2. **Action: Freeze** the weights and biases of the Feature Extractor (base layers) and only train the weights of a newly added, randomly initialized Classifier (head) for the new task.
3. **Rationale:** This significantly reduces the number of trainable parameters, making the model less complex and suitable for a small dataset. It assumes the pre-trained base layers have already learned excellent, generic feature representations that are transferable.

Fine-Tuning

1. **Process:** Used when the new dataset is closely related to the pre-trained data, but perhaps slightly different (e.g., newer models of cars).
2. **Action:** Unfreeze and train the weights and biases of the last few layers of the feature extractor, in addition to training the new classifier. The earliest layers, which learn only very generic features, are usually left frozen.
3. **Rationale:** This allows the model to adapt the most specific and complex feature representations to the subtle nuances of the new dataset, while retaining the generic feature knowledge learned by the initial layers.

6. Model Performance Metrics (Evaluation)

To evaluate a model's performance, several metrics are used, especially in binary and multi-class classification.

Metric	Formula	Description
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Overall proportion of correct predictions.
Precision	$TP / (TP + FP)$	Of all positive predictions, how many were correct.

Recall	$TP / (TP + FN)$	Of all actual positives, how many were correctly identified.
F1 Score	$(2 * Precision * Recall) / (Precision + Recall)$	The harmonic mean of precision and recall.

- **Multi-class Classification:** For problems with more than two classes, performance metrics are typically calculated by running a binary classification analysis for each class (one class as positive, all others as negative) and then taking a weighted average of the resulting per-class scores.